

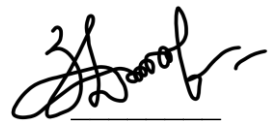
**КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА**

Факультет комп'ютерних наук та кібернетики
Кафедра теорії та технології програмування

Звіт до лабораторної роботи №1

**на тему: РОЗРОБЛЕННЯ ВЕБЗАСТОСУНКУ
ДЛЯ ПЛАНУВАННЯ РАЦІОНУ ХАРЧУВАННЯ**
з дисципліни «Інструментальні середовища та технології програмування»

Виконала студентка 2-го курсу
групи К-26
Злата ДОВБИК



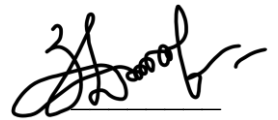
(підпис)

Науковий керівник:
доцент, кандидат фізико-математичних наук
Людмила ОМЕЛЬЧУК

(підпис)

Засвідчую, що в цій роботі немає запозичень
з праць інших авторів без відповідних посилань.

Студентка



(підпис)

Київ – 2026

РЕФЕРАТ

Обсяг роботи 36 сторінок, 6 рисунків, 19 таблиць, 10 джерел посилань, 3 додатки.

АВТЕНТИФІКАЦІЯ, БАЗА ДАНИХ, ВЕБЗАСТОСУНОК, ENTITY FRAMEWORK CORE, MEAL PLANNER, POSTGRESQL, ASP.NET CORE MVC, ПЛАНУВАННЯ ХАРЧУВАННЯ, РОЗРАХУНОК БЖВ, СПИСОК ПОКУПОК.

Об'єктом розроблення є процес автоматизації планування раціону харчування, розрахунку харчових цілей та формування списку покупок. Предметом розроблення є вебзастосунок Meal Planner, створений із використанням ASP.NET Core MVC, Entity Framework Core та PostgreSQL.

Метою роботи є проектування та реалізація вебзастосунку для планування меню за цільовими показниками білків, жирів і вуглеводів, збереження результатів планування, формування списку покупок, експорту та імпорту даних.

Методи розроблення: аналіз наявних цифрових рішень для планування харчування, моделювання предметної області, проектування реляційної бази даних, об'єктно-реляційне відображення, розроблення вебінтерфейсу за архітектурним шаблоном Model-View-Controller, ітеративне удосконалення функціональності. Інструменти розроблення: мова програмування C#, платформа .NET, ASP.NET Core MVC, Entity Framework Core, PostgreSQL, Docker, Razor Views, Bootstrap, ClosedXML, Google Charts.

Результатом роботи є повнофункціональний вебзастосунок Meal Planner, у якому реалізовано реєстрацію та вхід користувачів, розмежування доступу за ролями, конструктор меню на 2, 3 або 4 прийоми їжі, автоматизований розрахунок кількості продуктів за цільовими показниками БЖВ, збереження планів, формування списку покупок, експорт у текстовий та Excel-формати, імпорт Excel-файлів і візуалізацію результатів.

ЗМІСТ

СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАКИ	4
ВСТУП	5
1 ОГЛЯД НАЯВНИХ НА РИНКУ РІШЕНЬ	7
2 ОГЛЯД ВИКОРИСТАНИХ ТЕХНОЛОГІЙ	9
3 ПРИЗНАЧЕННЯ І ЦІЛІ СТВОРЕННЯ ВЕБЗАСТОСУНКУ	11
4 ВИМОГИ ДО ВЕБЗАСТОСУНКУ	14
5 ОПИС ОРГАНІЗАЦІЇ ІНФОРМАЦІЙНОЇ БАЗИ	18
6 РЕАЛІЗАЦІЯ ВЕБЗАСТОСУНКУ	22
7 ІНСТРУКЦІЯ КОРИСТУВАЧА	29
ВИСНОВКИ	32
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ	34
ДОДАТОК А СТРУКТУРА РЕПОЗИТОРІЮ ТА КОМАНДИ ЗАПУСКУ	35
ДОДАТОК Б USE CASE ДІАГРАМА ВЕБЗАСТОСУНКУ MEAL PLANNER	36
ДОДАТОК В ЛОГІЧНА СТРУКТУРА БАЗИ ДАНИХ MEAL PLANNER	37

СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАКИ

API — Application Programming Interface, програмний інтерфейс для взаємодії між програмними компонентами;

ASP.NET Core MVC — платформа та архітектурний підхід для створення вебзастосунків за шаблоном Model-View-Controller;

CRUD — Create, Read, Update, Delete, базові операції створення, перегляду, редагування та видалення даних;

EF Core — Entity Framework Core, структура об'єктно-реляційного відображення;

MVC — Model-View-Controller, архітектурний шаблон «модель — представлення — контролер»;

ORM — Object-Relational Mapping, об'єктно-реляційне відображення;

RBAC — Role-Based Access Control, керування доступом на основі ролей;

UI — User Interface, користувацький інтерфейс;

UX — User Experience, користувацький досвід;

БД — база даних;

БЖВ — білки, жири, вуглеводи;

СКБД — система керування базами даних;

У звіті назва Meal Planner використовується як власна назва розробленого вебзастосунку.

ВСТУП

Оцінка сучасного стану об'єкта розроблення. Планування харчування сьогодні все частіше переноситься у цифрове середовище, оскільки користувачам потрібно швидко поєднувати власні цілі, харчову цінність продуктів і практичні потреби на кілька днів. У межах такого процесу важливо не лише підрахувати калорійність, а й врахувати білки, жири, вуглеводи, підібрати продукти та сформувати список покупок. Автоматизований вебзастосунок дає змогу зменшити кількість ручних перерахунків і зробити планування раціону більш послідовним.

Актуальність роботи та підстави для її виконання. Під час складання меню користувач має враховувати цілі за білками, жирами та вуглеводами, кількість прийомів їжі, поживну цінність продуктів, одиниці вимірювання та період, на який потрібен список покупок. Meal Planner об'єднує ці дії в один сценарій: користувач вводить цілі, обирає продукти й отримує готове меню разом зі списком покупок.

Метою лабораторного проєкту є створення вебзастосунку Meal Planner для планування раціону харчування на основі цільових показників БЖВ, з можливістю збереження планів, формування списку покупок, експорту, імпорту та візуалізації результатів.

Для досягнення поставленої мети необхідно виконати такі завдання:

- проаналізувати наявні цифрові рішення для планування харчування та визначити корисні підходи до організації користувацького сценарію;
- спроектувати предметну область вебзастосунку Meal Planner та визначити основні сутності бази даних;
- реалізувати збереження даних за допомогою PostgreSQL та Entity Framework Core з підходом Code First;
- розробити користувацький інтерфейс конструктора меню з вибором цілей БЖВ, кількості прийомів їжі та продуктів;

- реалізувати автоматизований розрахунок меню, підсумкових показників та списку покупок;
- додати механізми автентифікації, авторизації та розмежування доступу за ролями;
- реалізувати експорт, імпорт і візуалізацію результатів.

Об'єктом розроблення є процес автоматизації планування раціону харчування, розрахунку харчових цілей та формування списку покупок. Предметом розроблення є програмна реалізація вебзастосунку Meal Planner на платформі ASP.NET Core MVC з використанням EF Core, PostgreSQL і допоміжних засобів для експорту та візуалізації.

Під час виконання роботи застосовано методи аналізу наявних рішень, моделювання предметної області, проєктування реляційної бази даних, розроблення програмного забезпечення за шаблоном MVC, а також тестування типових сценаріїв використання.

Результати роботи можуть бути використані як навчальний приклад створення вебзастосунку з базою даних, авторизацією, адміністративною частиною, користувацьким сценарієм, експортом даних і візуальною аналітикою.

1 ОГЛЯД НАЯВНИХ НА РИНКУ РІШЕНЬ

Огляд сервісів цифрового планування харчування

У сфері цифрового контролю харчування поширені застосунки, які допомагають користувачам відстежувати продукти, калорійність, макронутрієнти та індивідуальні цілі. До таких рішень належать MyFitnessPal [8], YAZIO [9], FatSecret [10] та інші сервіси, орієнтовані на ведення харчового щоденника і контроль раціону.

Більшість наявних рішень мають спільні функції: каталог продуктів, фіксацію прийомів їжі, підрахунок калорій і макронутрієнтів, історію харчування та профіль користувача. Водночас частина таких сервісів зосереджена саме на щоденному обліку, а не на швидкому формуванні готового меню та списку покупок за обраними продуктами.

Порівняльний аналіз рішень

Для визначення доцільних функцій Meal Planner було порівняно кілька типових рішень. Результати порівняння подано в таблиці 1.1.

Таблиця 1.1 — Порівняльна характеристика наявних рішень

Рішення	Основне призначення	Переваги	Особливості, враховані в Meal Planner
MyFitnessPal	ведення харчового щоденника та контроль калорійності	велика база продуктів, контроль макронутрієнтів, історія споживання	доцільність збереження харчових цілей і підрахунку БЖВ
YAZIO	планування харчування та контроль раціону	зручний користувацький інтерфейс, фокус на щоденному прогресі	необхідність простого сценарію для користувача без перевантаження формами

Рішення	Основне призначення	Переваги	Особливості, враховані в Meal Planner
FatSecret	облік харчування, продуктів і щоденних показників	каталог продуктів, щоденник, підрахунок поживних речовин	потреба у зрозумілому поданні підсумкових значень
Meal Planner	формування меню за ціллю БЖВ і створення списку покупок	вебінтерфейс, розрахунок кількості продуктів, експорт, імпорт, ролі	спрямованість на готовий результат: меню та список покупок

Висновки до розділу

Проведений огляд показав, що на ринку існують розвинені інструменти для обліку харчування, однак у межах лабораторного проєкту доцільно зосередитися на чіткому сценарії планування: користувач задає цілі, обирає продукти, отримує розраховане меню і список покупок. Саме така логіка покладена в основу вебзастосунку Meal Planner.

2 ОГЛЯД ВИКОРИСТАНИХ ТЕХНОЛОГІЙ

Обґрунтування вибору технологічного стеку

Для реалізації Meal Planner обрано технології, які добре поєднуються між собою та відповідають навчальним цілям лабораторного проєкту. Основу становить платформа .NET і вебшаблон ASP.NET Core MVC, що дає змогу розділити логіку обробки запитів, доменну модель і представлення сторінок [1].

Короткий опис використаного технологічного стеку наведено в таблиці 2.1.

Таблиця 2.1 — Використані технології та їх призначення

Технологія	Призначення у вебзастосунку
C# та .NET	реалізація серверної логіки, моделей, контролерів і допоміжних методів
ASP.NET Core MVC	обробка HTTP-запитів, маршрутизація, контролери та Razor-представлення
Entity Framework Core	об'єктно-реляційне відображення та робота з базою даних через DbContext
PostgreSQL	реляційне збереження даних користувачів, продуктів, планів, прийомів їжі та списків покупок
Docker Compose	локальне розгортання PostgreSQL у контейнері
Razor Views і Bootstrap	створення україномовного користувацького інтерфейсу
ClosedXML	експорт та імпорт Excel-файлів
Google Charts	побудова діаграм для аналізу результатів

Технології серверної частини

Серверна частина реалізована на основі ASP.NET Core MVC. Такий підхід дозволяє виділити контролери, які приймають запити користувача, моделі, які описують дані й бізнес-об'єкти, та представлення, які відповідають за відображення HTML-сторінок. У проєкті це відображено через окремі контролери для планувальника, авторизації, довідкових сутностей та аналітичних API.

Для керування доступом використано механізми автентифікації та авторизації ASP.NET Core [3]. У Meal Planner реалізовано cookie-автентифікацію, збереження хешованих паролів і claims-based авторизацію з ролями Admin та User.

Технології роботи з даними

Для роботи з базою даних застосовано Entity Framework Core, який дозволяє описувати таблиці бази даних через класи C# і конфігурацію Fluent API [2]. У проєкті використано підхід Code First, за якого структура бази формується на основі доменних сутностей та міграцій.

Як СКБД використано PostgreSQL, розгорнутий у Docker-контейнері. PostgreSQL забезпечує підтримку реляційної моделі, зовнішніх ключів, індексів, обмежень цілісності та стабільної роботи з пов'язаними даними [4]. Docker Compose спрощує запуск середовища розроблення, оскільки конфігурація бази даних описана в одному файлі [5].

Технології інтерфейсу, експорту та візуалізації

Користувацький інтерфейс реалізовано за допомогою Razor Views, HTML, CSS і Bootstrap. Інтерфейс орієнтований на роботу з кінцевим користувачем: головною точкою входу є конструктор меню, а адміністративні сторінки виконують допоміжну роль керування довідковими даними.

Для експорту та імпорту Excel-файлів використано бібліотеку ClosedXML, яка дозволяє створювати структуровані аркуші з підсумками, меню та списком покупок [6]. Для візуалізації результатів використано Google Charts, що дає змогу будувати діаграми розподілу калорій та макронутрієнтів без додаткової серверної обробки графіки [7].

Отже, обраний стек технологій забезпечує повний цикл роботи вебзастосунку: від взаємодії користувача з формами до збереження результатів у базі даних, експорту файлів і візуального аналізу.

3 ПРИЗНАЧЕННЯ І ЦІЛІ СТВОРЕННЯ ВЕБЗАСТОСУНКУ

Призначення вебзастосунок

Призначенням вебзастосунок Meal Planner є автоматизація процесу формування меню на основі цільових показників білків, жирів і вуглеводів. Вебзастосунок дозволяє користувачу задати харчові цілі, вибрати кількість прийомів їжі, обрати продукти за категоріями, отримати розраховані кількості продуктів, переглянути підсумки та сформувати список покупок.

Meal Planner також підтримує адміністративне керування довідковими даними: користувачами, продуктами, одиницями вимірювання, планами, прийомами їжі, списками покупок та експортами. Це дає змогу розглядати вебзастосунок не лише як форму розрахунку, а як цілісне програмне рішення для збереження та повторного використання даних.

Актори та основні сценарії використання

Основні актори та сценарії використання вебзастосунок описано в цьому розділі. Use Case діаграму вебзастосунок Meal Planner винесено в додаток Б.

У вебзастосунку виділено два типи акторів: користувач та адміністратор. Користувач виконує основний сценарій планування меню, а адміністратор має доступ до керування довідковими сутностями й перегляду службової інформації.

Опис акторів вебзастосунок наведено в таблиці 3.1.

Таблиця 3.1 — Актори вебзастосунок Meal Planner

Актор	Опис	Основні можливості
Користувач	zareestrovaniy користувач, який планує харчування	введення цілей БЖВ, вибір продуктів, розрахунок меню, збереження плану, експорт та імпорт результатів
Адміністратор	користувач із розширеними правами доступу	керування продуктами, одиницями вимірювання, користувачами, планами, списками покупок, експортами й аналітичними даними

Цілі та завдання роботи

Основною ціллю створення Meal Planner є надання користувачу зручного інструменту для побудови меню за власними харчовими цілями. Застосунок має зменшити кількість ручних розрахунків і зробити процес планування більш послідовним.

У межах лабораторного проекту поставлено такі прикладні та навчальні завдання:

- побудувати доменну модель предметної області планування харчування;
- створити базу даних для збереження користувачів, продуктів, цілей, планів і списків покупок;
- реалізувати основний користувацький сценарій у вигляді конструктора меню;
- забезпечити розрахунок кількості продуктів відповідно до обраних цілей БЖВ;
- підготувати функції експорту, імпорту та візуалізації результатів;
- організувати захищений доступ до сторінок вебзастосунку.

Функціонал мінімально життєздатної та розширеної версії

Функціональність Meal Planner поділено на мінімально необхідну та розширену. Такий поділ дозволяє показати пріоритетність реалізації, що наведено в таблиці 3.2.

Таблиця 3.2 — Пріоритетність реалізації функціональності

Рівень	Функціональність	Призначення
MVP	створення користувача, збереження продуктів і цілей БЖВ	базове збереження даних предметної області
MVP	розрахунок меню за обраними продуктами	отримання основного результату роботи вебзастосунку

Рівень	Функціональність	Призначення
MVP	формування списку покупок	перетворення меню на практичний список продуктів
Розширений	автентифікація та ролі	обмеження доступу до сторінок і відокремлення адміністративних функцій
Розширений	експорт TXT та Excel	збереження результатів у файлах для подальшого використання
Розширений	імпорт Excel	перевірка та повторне використання сформованих списків покупок
Розширений	діаграми та статистика	наочний аналіз результатів планування

4 ВИМОГИ ДО ВЕБЗАСТОСУНКУ

Вимоги до вебзастосунок в цілому

Meal Planner повинен забезпечувати користувачу повний сценарій роботи з меню: від входу до вебзастосунок до отримання готового меню, списку покупок та файлів експорту. Вебзастосунок має бути зрозумілим для користувача, підтримувати україномовний інтерфейс і надавати повідомлення про помилки в межах загального дизайну.

Загальні вимоги до вебзастосунок наведено в таблиці 4.1.

Таблиця 4.1 — Загальні вимоги до вебзастосунок

Вимога	Опис
Цілісність даних	усі основні дані зберігаються в PostgreSQL, зв'язки між сутностями налаштовані через зовнішні ключі та правила видалення
Зручність інтерфейсу	основний сценарій роботи винесено на сторінку конструктора меню
Захищеність доступу	сторінки вебзастосунок доступні після автентифікації, адміністративні функції обмежені роллю Admin
Локалізація	основні назви, кнопки, повідомлення і пояснення подані українською мовою
Розширюваність	структура проєкту поділена на Domain, Infrastructure та Web, що спрощує подальший розвиток

Підсистема користувача

Підсистема користувача забезпечує основний сценарій роботи Meal Planner. Перелік функцій цієї підсистеми наведено в таблиці 4.2.

Таблиця 4.2 — Функції підсистеми користувача

Функція	Завдання
Реєстрація та вхід	створення облікового запису, перевірка даних входу, відкриття доступу до конструктора меню
Введення цілей БЖВ	задання або підтягнення цільових значень білків, жирів і вуглеводів
Налаштування прийомів їжі	вибір 2, 3 або 4 прийомів їжі та задання назв прийомів
Вибір продуктів	вибір білкових, вуглеводних і жирових продуктів через модальні списки
Розрахунок меню	визначення кількості продуктів і підсумкових значень БЖВ та калорій
Збереження результату	створення плану харчування і пов'язаного списку покупок
Експорт та імпорт	завантаження результатів у TXT або Excel, а також читання Excel-файлу зі списком покупок

Підсистема адміністратора

Підсистема адміністратора призначена для керування даними, які підтримують роботу основного сценарію. Вона не замінює користувацький конструктор меню, а доповнює його службовими можливостями. Основні функції адміністратора наведено в таблиці 4.3.

Таблиця 4.3 — Функції підсистеми адміністратора

Функція	Завдання
Керування користувачами	перегляд і супровід облікових записів користувачів
Керування продуктами	додавання, редагування та видалення продуктів, категорій, харчових показників і прив'язаних іконок
Керування одиницями вимірювання	підтримка одиниць маси, об'єму та кількості для відображення продуктів

Функція	Завдання
Керування планами	перегляд створених планів, прийомів їжі та позицій меню
Керування списками покупок	перегляд списків покупок і позицій, сформованих за планами
Перегляд експортів	контроль сформованих файлів експорту

Технічні та нефункціональні вимоги

Вебзастосунок має запускатися у середовищі .NET із підключенням до PostgreSQL. База даних розгортається локально за допомогою Docker Compose, а підключення до неї виконується через рядок підключення в конфігураційному файлі вебпроєкту.

Технічні та нефункціональні вимоги подано в таблиці 4.4.

Таблиця 4.4 — Технічні та нефункціональні вимоги

Категорія	Вимога
Операційна система	Windows 10/11 або інше середовище, у якому доступні .NET SDK і Docker
Платформа виконання	.NET 9
СКБД	PostgreSQL, запущений у Docker-контейнері
Браузери	сучасні версії Google Chrome, Microsoft Edge, Mozilla Firefox, Opera або Safari
Безпека	хешування паролів, cookie-автентифікація, перевірка ролей, антифальсифікаційні токени у формах
Валідація	перевірка обов'язкових полів і повідомлення про помилки українською мовою
Підтримувані формати файлів	експорт TXT і XLSX, імпорт XLSX

Таким чином, вимоги охоплюють як основні користувацькі операції, так і службову підтримку даних, захищений доступ та можливість подальшого розвитку вебзастосунку.

5 ОПИС ОРГАНІЗАЦІЇ ІНФОРМАЦІЙНОЇ БАЗИ

Вибір системи керування базами даних

Для збереження даних Meal Planner використано реляційну СКБД PostgreSQL [4]. Вибір реляційної моделі є доцільним, оскільки предметна область містить багато пов'язаних сутностей: користувачі мають профілі, макроцілі, плани, плани містять прийоми їжі, прийоми їжі містять позиції продуктів, а списки покупок формуються на основі збережених планів.

PostgreSQL запускається у контейнері Docker, що спрощує налаштування однакового середовища розроблення. Підключення до бази даних виконується через конфігураційний файл appsettings.json.

Логічна структура бази даних

Логічну структуру бази даних Meal Planner наведено в додатку В на рисунку В.1.

Діаграма відображає основні сутності та зв'язки між ними. Центральними таблицями є users, food, macro, plan, meal, meal_item, shop_list і shop_item. Окремо виділено таблицю auth_account, яка забезпечує зберігання автентифікаційних даних та ролі користувача.

Перелік таблиць бази даних

Повний перелік основних таблиць бази даних та їх призначення наведено в таблиці 5.1.

Таблиця 5.1 — Перелік таблиць бази даних Meal Planner

Таблиця	Призначення
users	зберігає базові дані користувачів: ім'я, електронну пошту та дату створення
auth_account	зберігає хеш пароля, роль і зв'язок з користувачем
profile	зберігає профіль користувача, зокрема вагу та ціль
macro	зберігає цільові значення білків, жирів і вуглеводів

Таблиця	Призначення
configuration	зберігає активні налаштування користувача, кількість прийомів і активну макроціль
macro_log	фіксує історію змін макроцілей
icon	зберігає коди та емоїї-іконки продуктів
unit	зберігає одиниці вимірювання продуктів
food	зберігає продукти, категорії та харчову цінність на 100 одиниць
plan	зберігає сформовані плани харчування
meal	зберігає прийоми їжі в межах плану
meal_item	зберігає продукти у прийомах їжі, кількість і копію харчових значень
shop_list	зберігає список покупок, сформований для плану
shop_item	зберігає позиції списку покупок
export	зберігає інформацію про сформовані файли експорту

Опис основних таблиць

Для розуміння організації інформаційної бази в таблиці 5.2 подано опис ключових атрибутів основних таблиць.

Таблиця 5.2 — Основні атрибути таблиць бази даних

Таблиця	Атрибут	Тип даних	Опис
users	id	uuid	первинний ключ користувача
users	email	text	унікальна електронна пошта користувача
users	name	text	ім'я користувача
auth_account	password_hash	varchar(512)	хеш пароля користувача
auth_account	role	varchar(32)	роль користувача в системі

Таблиця	Атрибут	Тип даних	Опис
macro	protein_g	numeric	цільова кількість білків у грамах
macro	carbs_g	numeric	цільова кількість вуглеводів у грамах
macro	fat_g	numeric	цільова кількість жирів у грамах
food	category	enum/int	категорія продукту: білковий, вуглеводний або жировий
food	protein_per_100	numeric	білки на 100 одиниць продукту
food	carbs_per_100	numeric	вуглеводи на 100 одиниць продукту
food	fat_per_100	numeric	жири на 100 одиниць продукту
plan	days	integer	кількість днів, на які формується список покупок
meal	meal_no	integer	номер прийому їжі
meal_item	quantity_value	numeric	розрахована кількість продукту
shop_item	total_quantity_value	numeric	загальна кількість продукту для списку покупок
export	file_url	text	шлях до сформованого файлу експорту

Особливістю моделі є таблиця `meal_item`, у якій зберігається не лише посилання на продукт, а й копія харчової цінності на момент створення плану. Це робить збережені плани стабільними навіть у разі подальшого редагування продукту в довіднику.

У таблиці 5.3 подано основні типи зв'язків між сутностями.

Таблиця 5.3 — Основні зв'язки між таблицями

Зв'язок	Тип	Призначення
users — profile	1:1	кожен користувач може мати один профіль
users — macro	1:M	користувач може мати кілька записів макроцілей
users — food	1:M	користувач може мати власні продукти або працювати з початковим каталогом
plan — meal	1:M	один план містить кілька прийомів їжі
meal — meal_item	1:M	один прийом містить кілька позицій продуктів
plan — shop_list	1:M	для плану може бути сформовано список покупок
shop_list — shop_item	1:M	список покупок містить окремі позиції продуктів
unit — food	1:M	одиниця вимірювання використовується для опису харчової цінності продуктів

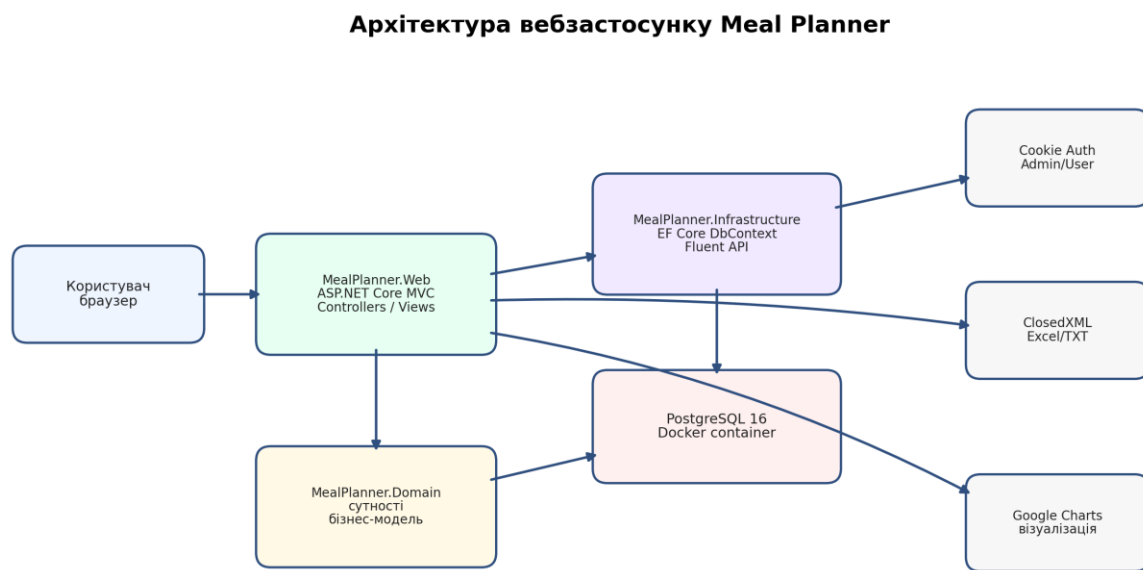
Наведена структура забезпечує повноцінне збереження як довідкових даних, так і результатів користувацького планування.

6 РЕАЛІЗАЦІЯ ВЕБЗАСТОСУНКУ

Загальна архітектура програмного рішення

Проект Meal Planner організовано за багатошаровою структурою. Основні частини рішення — MealPlanner.Domain, MealPlanner.Infrastructure та MealPlanner.Web. Такий поділ дозволяє відокремити доменні сутності, доступ до даних та вебінтерфейс.

Загальну архітектуру програмного рішення наведено на рисунку 6.1.



Шари розділено так, щоб доменна модель не залежала від інтерфейсу, а доступ до БД був зосереджений в інфраструктурному шарі.

Рисунок 6.1 — Архітектура вебзастосунку Meal Planner

Шар Domain містить сутності предметної області. Шар Infrastructure містить контекст бази даних, конфігурацію таблиць і початкове наповнення даних. Шар Web містить контролери, моделі представлення, Razor-сторінки, стилі та логіку взаємодії з користувачем.

Структуру основних проєктів наведено в таблиці 6.1.

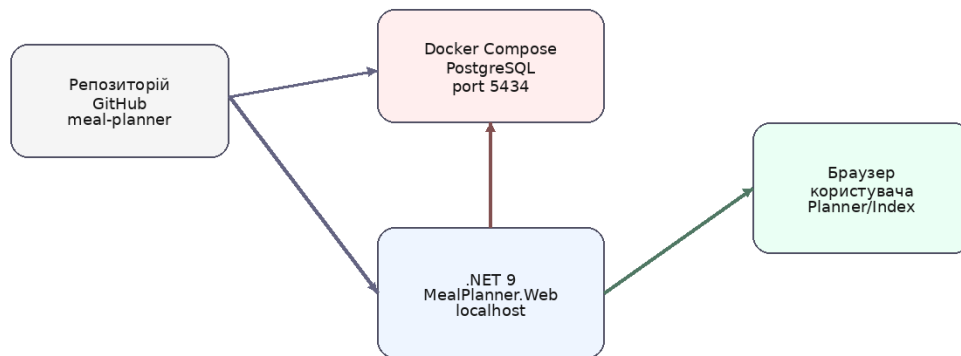
Таблиця 6.1 — Структура програмного рішення

Проект або каталог	Призначення
MealPlanner.Domain	сутності предметної області: користувачі, продукти, плани, прийоми їжі, списки покупок, експорти
MealPlanner.Infrastructure	контекст бази даних, конфігурація Fluent API, міграції та початкове наповнення даних
MealPlanner.Web	ASP.NET Core MVC-вебзастосунок: контролери, представлення, моделі сторінок, стилі, авторизація
docker-compose.yml	опис контейнера PostgreSQL та параметрів локальної бази даних
appsettings.json	рядок підключення до бази даних

Налаштування середовища та запуск

Локальне розгортання вебзастосунку виконується за допомогою Docker Compose та .NET SDK. Схему локального запуску наведено на рисунку 6.2.

Схема локального розгортання



Після запуску контейнера з базою даних вебзастосунок застосовує міграції, заповнює початкові дані й відкриває користувацький інтерфейс конструктора меню.

Рисунок 6.2 — Схема локального розгортання Meal Planner

Після запуску контейнера PostgreSQL вебзастосунок підключається до бази даних, застосовує міграції та заповнює початковий каталог продуктів. Це спрощує перевірку роботи, оскільки користувач одразу отримує готові довідкові дані для розрахунку меню.

Основні команди запуску наведено в таблиці 6.2.

Таблиця 6.2 — Команди запуску вебзастосунку

Команда	Призначення
<code>docker compose up -d</code>	запуск контейнера PostgreSQL
<code>dotnet restore .\app\MealPlanner.sln</code>	відновлення пакетів NuGet
<code>dotnet build .\app\MealPlanner.sln</code>	збирання програмного рішення
<code>dotnet ef database update --project .\app\MealPlanner.Infrastructure\MealPlanner.Infrastructure.csproj --startup-project .\app\MealPlanner.Web\MealPlanner.Web.csproj</code>	застосування міграцій до бази даних
<code>dotnet run --project .\app\MealPlanner.Web\MealPlanner.Web.csproj</code>	запуск вебзастосунку

Реалізація доменної моделі та доступу до даних

Доменна модель представлена класами сутностей. Наприклад, сутність `Food` описує продукт, його категорію, харчову цінність на 100 одиниць, одиницю вимірювання та іконку. Сутності `Plan`, `Meal` і `MealItem` описують результат планування: план, прийоми їжі та продукти в кожному прийомі.

Доступ до даних реалізовано через `MealPlannerDbContext`. У ньому оголошено `DbSet` для кожної сутності та налаштовано назви таблиць, колонок, індекси, зовнішні ключі й поведінку видалення. Це забезпечує узгодженість структури бази даних із доменною моделлю.

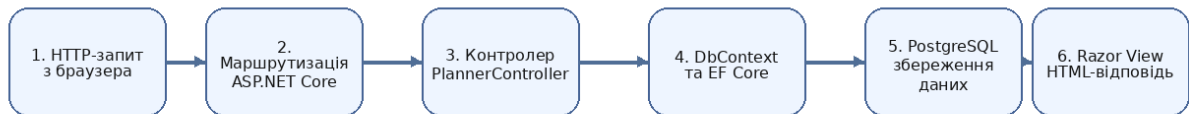
Після запуску вебзастосунку виконується початкове наповнення бази даних. Seed-логіка створює демонстраційного користувача, одиниці вимірювання, іконки, макроцілі та каталог продуктів. Завдяки цьому вебзастосунок готовий до демонстрації відразу після запуску.

Реалізація конструктора меню

Основним користувацьким модулем є конструктор меню. Він дозволяє вибрати кількість прийомів їжі, задати цілі БЖВ, обрати білкові, вуглеводні та жирові продукти для кожного прийому, після чого отримати розраховані кількості продуктів.

Послідовність обробки запиту користувача наведено на рисунку 6.3.

Послідовність обробки запиту користувача



Приклад сценарію: користувач вводить цілі БЖВ, обирає продукти та натискає кнопку розрахунку. Контролер перевіряє дані, отримує продукти з БД, виконує розподіл макронутрієнтів і повертає сторінку з результатом.

Рисунок 6.3 — Послідовність обробки запиту користувача

У PlannerController реалізовано дії для відкриття сторінки, підтягнення цілей користувача, розрахунку меню, збереження плану, експорту, імпорту та формування Excel-файлів. Дані для сторінки передаються через PlannerPageViewModel, яка містить цілі користувача, вибрані продукти, результати розрахунку, список покупок та імпортовані позиції.

Основні операції конструктора меню подано в таблиці 6.3.

Таблиця 6.3 — Операції конструктора меню

Операція	Опис результату
LoadDefaults	підтягує активні цілі БЖВ і кількість прийомів їжі з конфігурації користувача
Calculate	перевіряє введені дані, отримує продукти з бази даних і формує результат розрахунку
Save	створює запис плану, прийомів їжі, позицій меню, списку покупок та позицій списку покупок
ExportText	формує текстовий файл з меню та списком покупок
ExportExcel	створює Excel-файл із аркушами «Підсумок», «Меню» і «Список покупок»
ImportExcel	зчитує Excel-файл і відображає імпортований список покупок

Реалізація автентифікації та розмежування доступу

У Meal Planner реалізовано реєстрацію, вхід і вихід користувачів. Під час реєстрації створюється користувач і пов'язаний запис AuthAccount. Пароль не зберігається у відкритому вигляді: для нього формується хеш. Після успішного входу створюються claims із ідентифікатором користувача, ім'ям, електронною поштою та роллю.

Перший зареєстрований користувач отримує роль Admin, наступні — роль User. Це дає змогу швидко створити адміністративний обліковий запис під час першого запуску. Доступ до службових контролерів обмежено роллю Admin через спеціальну конвенцію контролерів.

Таблиця 6.4 — Реалізовані механізми захисту

Механізм	Реалізація
Хешування паролів	використано PasswordHasher для AuthAccount
Cookie-автентифікація	налаштовано схему MealPlanner.Auth

Механізм	Реалізація
Claims	збережено ідентифікатор, ім'я, електронну пошту та роль користувача
Ролі	підтримуються ролі Admin і User
Захист форм	POST-дії використовують антифальсифікаційні токени
Адміністративний доступ	службові контролери доступні лише користувачу з роллю Admin

Реалізація експорту, імпорту та аналітики

Експорт результатів реалізовано у двох форматах: TXT і XLSX. Текстовий експорт зручний для швидкого копіювання меню, а Excel-експорт формує структурований файл з окремими аркушами. Excel-файл містить підсумкові цілі, фактичні значення, продукти за прийомами їжі та список покупок.

Імпорт Excel-файлів дозволяє завантажити список покупок у форматі XLSX та переглянути його безпосередньо на сторінці конструктора. У разі некоректного формату користувач отримує зрозуміле повідомлення про помилку.

Для аналітики реалізовано візуалізацію результатів планування: діаграму розподілу калорій по прийомах їжі та діаграму розподілу БЖВ. Окремий API-контролер надає дані для статистики продуктів за категоріями.

Перевірка коректності роботи

Перевірку роботи вебзастосунку виконано через основні сценарії користувача й адміністратора. Особливу увагу приділено валідації форм, коректності розрахунку меню, збереженню планів, створенню списку покупок та перевірці експорту й імпорту.

Таблиця 6.5 — Основні перевірені сценарії

Сценарій	Очікуваний результат
реєстрація нового користувача	створюється користувач і автентифікаційний запис

Сценарій	Очікуваний результат
вхід із правильними даними	користувач переходить до конструктора меню
вхід із неправильним паролем	відображається повідомлення про помилку
розрахунок без цілей БЖВ	форма показує повідомлення про необхідність заповнення цілей
розрахунок з вибраними продуктами	відображається меню, підсумки БЖВ і список покупок
збереження плану	у базі створюються план, прийоми їжі, позиції меню, список покупок
експорт Excel	завантажується XLSX-файл із трьома аркушами
імпорт неправильного формату	відображається повідомлення про підтримку лише .xlsx

Результати перевірки підтверджують, що вебзастосунок виконує основні функції планування харчування та коректно реагує на типові некоректні дії користувача.

7 ІНСТРУКЦІЯ КОРИСТУВАЧА

Вхід до вебзастосунку

Для початку роботи користувач відкриває вебзастосунок у браузері. Якщо користувач не автентифікований, він переходить на сторінку входу або реєстрації. Після успішного входу відкривається сторінка конструктора меню.

Основний користувацький сценарій наведено на рисунку 7.1.



Сценарій завершується отриманням готового меню, підсумкових показників БЖВ, списку покупок на вибрану кількість днів і файлів для збереження результатів.

Рисунок 7.1 — Основний користувацький сценарій Meal Planner

Сценарій побудований так, щоб користувач послідовно виконав усі необхідні дії: увійшов до вебзастосунку, підтягнув або ввів цілі, обрав прийоми їжі та продукти, розрахував меню і отримав список покупок.

Формування меню та списку покупок

На сторінці конструктора користувач обирає кількість прийомів їжі: 2, 3 або 4. Далі він вводить або підтягує власні цілі за білками, жирами та вуглеводами. Для кожного прийому їжі користувач може обрати білковий, вуглеводний та жировий продукт.

Після натискання кнопки розрахунку вебзастосунок формує готове меню. У результатах показано кількість кожного продукту, підсумкові білки, жири, вуглеводи та калорії для кожного прийому і для всього дня.

За потреби користувач задає кількість днів для списку покупок. Вебзастосунок автоматично підсумовує кількість продуктів і відображає зведений список.

Таблиця 7.1 — Кроки користувача під час формування меню

Крок	Дія користувача	Результат
1	обрати кількість прийомів їжі	активуються потрібні картки прийомів
2	ввести або підтягнути цілі БЖВ	цілі використовуються для розрахунку
3	обрати продукти у модальних списках	вибрані продукти зберігаються у формі
4	натиснути «Розрахувати меню»	відображається готове меню та підсумки
5	обрати кількість днів	формується список покупок на відповідний період
6	натиснути «Зберегти план і список покупок»	результат зберігається в базі даних

Експорт, імпорт і робота адміністратора

Після розрахунку меню користувач може експортувати результат у текстовий файл або Excel-файл. Excel-файл зручний для подальшого перегляду, редагування або друку, оскільки містить структуровані аркуші з підсумком, меню і списком покупок.

Функція імпорту дозволяє вибрати файл .xlsx і зчитати зі нього позиції списку покупок. Якщо файл має неправильний формат або не містить необхідних даних, вебзастосунок повідомляє про це користувача.

Адміністратор після входу може відкривати службові сторінки для керування продуктами, користувачами, одиницями вимірювання, планами, списками покупок та експортами. Це дозволяє підтримувати дані вебзастосунку в актуальному стані.

Формати файлів, які використовуються у вебзастосунку, наведено в таблиці 7.2.

Таблиця 7.2 — Формати експорту та імпорту

Формат	Напрямок	Призначення
TXT	експорт	швидке збереження текстового опису меню та списку покупок
XLSX	експорт	структурований файл з аркушами «Підсумок», «Меню», «Список покупок»
XLSX	імпорт	зчитування списку покупок із Excel-файлу

Таким чином, користувач отримує не лише розраховане меню, а й практичні інструменти для збереження, перегляду, перенесення та повторного використання результатів.

ВИСНОВКИ

У межах лабораторного проєкту було розроблено вебзастосунок Meal Planner для планування раціону харчування. Мета роботи досягнута: створено програмне рішення, яке дозволяє користувачу вводити або підтягувати цілі БЖВ, обирати продукти, розраховувати меню, формувати список покупок, зберігати результати та експортувати їх у файли.

У процесі виконання роботи виконано поставлені завдання: проаналізовано наявні рішення для цифрового планування харчування; спроектовано предметну область і структуру бази даних; реалізовано збереження даних у PostgreSQL за допомогою Entity Framework Core; створено користувацький інтерфейс конструктора меню; реалізовано розрахунок меню та списку покупок; додано механізми автентифікації, авторизації, експорту, імпорту та візуалізації результатів.

Під час виконання роботи проаналізовано наявні рішення у сфері цифрового планування харчування, визначено основні сценарії використання, спроектовано доменну модель і реляційну базу даних. Для збереження даних використано PostgreSQL, а для доступу до них — Entity Framework Core з підходом Code First.

У вебзастосунку реалізовано багат шарову структуру з окремими проєктами Domain, Infrastructure і Web. Такий підхід забезпечує зрозумілий поділ відповідальності: доменний шар описує предметну область, інфраструктурний шар відповідає за базу даних, а вебшар — за контролери, представлення, авторизацію та взаємодію з користувачем.

Особливу увагу приділено основному користувацькому сценарію. Конструктор меню дозволяє працювати не з окремими службовими таблицями, а з цілісним процесом планування: користувач задає цілі, обирає кількість прийомів їжі, вибирає продукти, отримує готовий результат і список покупок.

Також реалізовано cookie-автентифікацію з хешуванням паролів і розмежуванням доступу за ролями, експорт результатів у TXT та XLSX, імпорт

Excel-файлів, початкове наповнення каталогу продуктів і візуалізацію результатів за допомогою діаграм.

Під час розроблення поглиблено практичні навички роботи з ASP.NET Core MVC, Entity Framework Core, PostgreSQL, Docker, Razor Views, ClosedXML і механізмами автентифікації в ASP.NET Core. Створений вебзастосунок має зрозумілу архітектуру та може бути розширений у майбутньому, наприклад, додаванням персональних шаблонів меню, детальнішої аналітики або додаткових форматів експорту.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. ASP.NET Core MVC Overview. Microsoft Learn. URL: <https://learn.microsoft.com/aspnet/core/mvc/overview> (дата звернення: 26.04.2026)
2. Entity Framework Core Documentation. Microsoft Learn. URL: <https://learn.microsoft.com/ef/core/> (дата звернення: 26.04.2026)
3. Authentication and authorization in ASP.NET Core. Microsoft Learn. URL: <https://learn.microsoft.com/aspnet/core/security/> (дата звернення: 26.04.2026)
4. PostgreSQL 16 Documentation. PostgreSQL Global Development Group. URL: <https://www.postgresql.org/docs/16/> (дата звернення: 26.04.2026)
5. Docker Compose Documentation. Docker Docs. URL: <https://docs.docker.com/compose/> (дата звернення: 26.04.2026)
6. ClosedXML Documentation. URL: <https://docs.closedxml.io/> (дата звернення: 26.04.2026)
7. Google Charts Documentation. Google for Developers. URL: <https://developers.google.com/chart> (дата звернення: 26.04.2026)
8. MyFitnessPal. URL: <https://www.myfitnesspal.com/> (дата звернення: 26.04.2026)
9. YAZIO. URL: <https://www.yazio.com/> (дата звернення: 26.04.2026)
10. FatSecret. URL: <https://www.fatsecret.com/> (дата звернення: 26.04.2026)

ДОДАТОК А

СТРУКТУРА РЕПОЗИТОРІЮ ТА КОМАНДИ ЗАПУСКУ

Код вебзастосунку розміщено у репозиторії GitHub: <https://github.com/zlat1x/meal-planner>.

Основна структура репозиторію наведена в таблиці А.1.

Таблиця А.1 — Структура репозиторію Meal Planner

Шлях	Призначення
app/MealPlanner.sln	файл рішення Visual Studio / .NET
app/MealPlanner.Domain	доменний шар із сутностями
app/MealPlanner.Infrastructure	інфраструктурний шар із DbContext, міграціями та початковими даними
app/MealPlanner.Web	вебшар ASP.NET Core MVC
docker-compose.yml	конфігурація PostgreSQL-контейнера
db/schema.sql	початковий SQL-артефакт для першого етапу роботи
docs	документаційні матеріали проєкту

Типова послідовність запуску вебзастосунку в локальному середовищі:

- відкрити кореневу папку репозиторію;
- запустити контейнер бази даних командою `docker compose up -d`;
- відновити пакети та зібрати рішення командами `dotnet restore` і `dotnet build`;
- застосувати міграції до бази даних;
- запустити вебпроєкт `MealPlanner.Web`;
- відкрити вебзастосунок у браузері та перейти до конструктора меню.

Додаток містить лише службову інформацію для відтворення запуску вебзастосунку та не дублює основні розділи звіту.

ДОДАТОК Б

USE CASE ДІАГРАМА ВЕБЗАСТОСУНКУ MEAL PLANNER

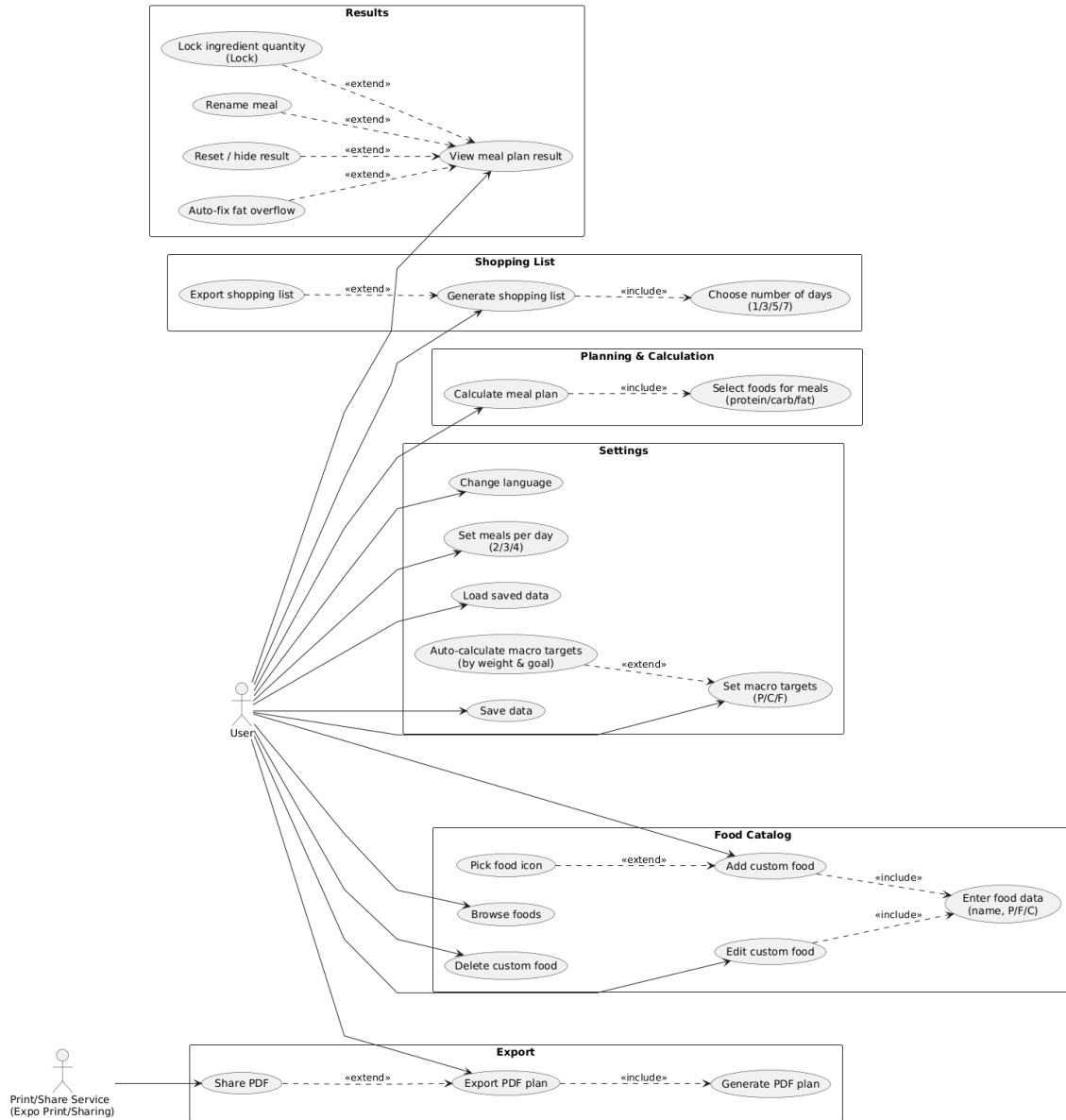


Рисунок Б.1 — Use Case діаграма вебзастосунку Meal Planner

ДОДАТОК В

ЛОГІЧНА СТРУКТУРА БАЗИ ДАНИХ ВЕБЗАСТОСУНКУ MEAL PLANNER

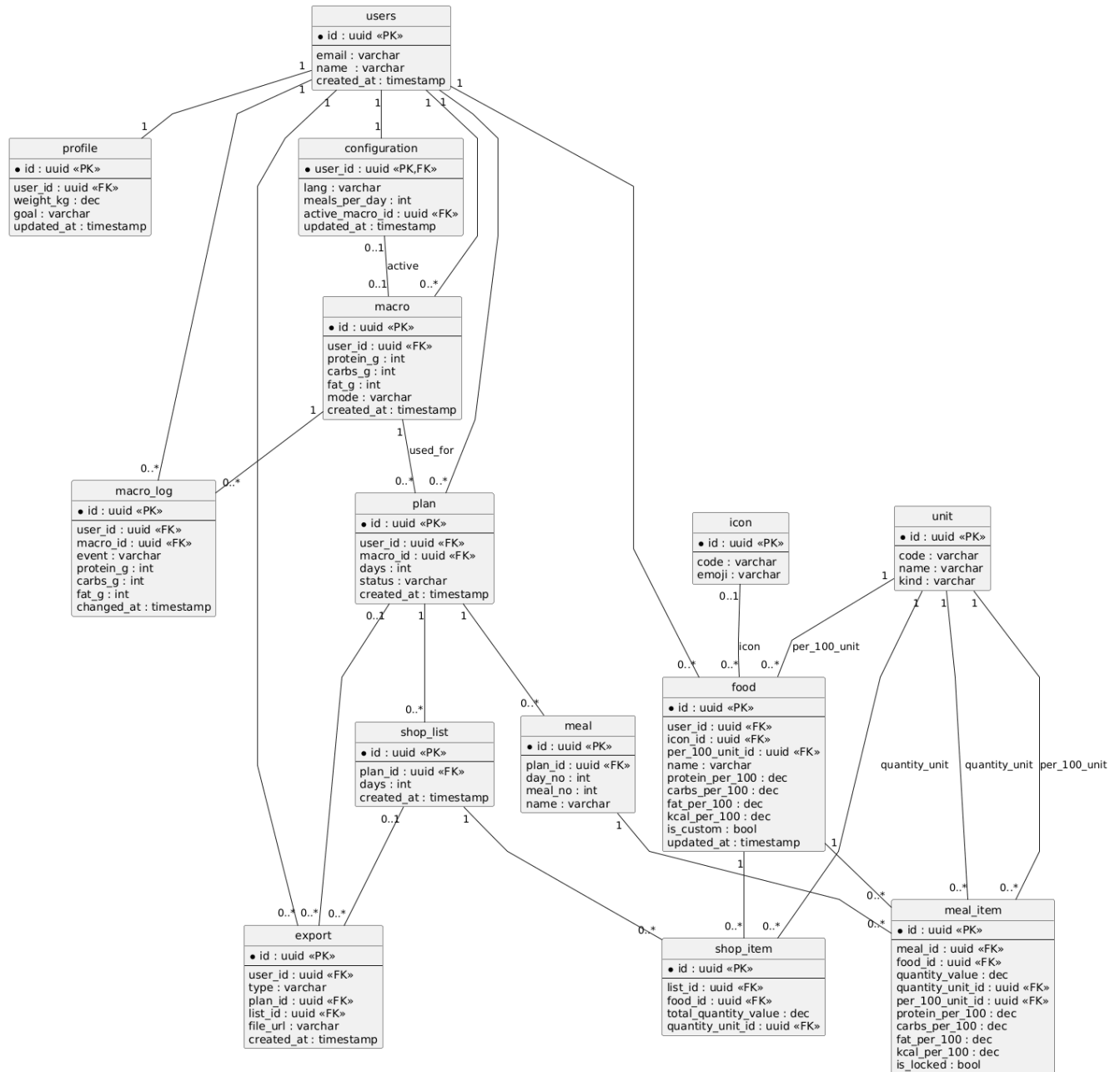


Рисунок В.1 — Логічна структура бази даних Meal Planner